

**Smart Farmer Procurement and Queue Management
System
(AgriQueue)**

A

Project-I Report

Submitted

In Partial Fulfilment
For the award of the Degree of
**Bachelor of Technology
(VII Semester)**



Supervised By:

Dr. / Mr. [Guide Name]
[Designation]

Submitted By:

[Student Name]
Roll No: [Roll No]

**Department of Computer Science and Engineering
Jodhpur Institute of Engineering and Technology(Autonomous)
JIET Universe, Jodhpur
Session 2026-27**

Certificate

This is to certify that the project entitled “**Smart Farmer Procurement and Queue Management System (AgriQueue)**” has been carried out by the students of **Jodhpur Institute of Engineering & Technology, Jodhpur** under my guidance and supervision in partial fulfillment of the degree of Bachelor of Technology in Computer Science and Engineering of Bikaner Technical University during the academic year of 2026-27.

The project work is original and has not been submitted elsewhere for the award of any degree or diploma.

Date:

Place: Jodhpur

Supervisor’s Name

[Designation]

Acknowledgment

We express our sincere gratitude to all those who have been instrumental in the completion of this project report on “*Smart Farmer Procurement and Queue Management System (AgriQueue)*”, which represents a significant step toward digital transformation in Indian agricultural procurement.

We are deeply thankful to our project guide **[Guide Name]**, *[Designation]*, Department of Computer Science and Engineering, JIET, Jodhpur, for the invaluable guidance, continuous encouragement, and insightful feedback throughout the project development process. Their expertise in software engineering and web technologies has been the cornerstone of this project.

We extend special gratitude to the **Head of Department**, Department of Computer Science and Engineering, for providing all necessary facilities and an encouraging academic environment that enabled us to work on this innovative project.

We are thankful to the **Jodhpur Institute of Engineering and Technology (Autonomous)**, JIET Universe, Jodhpur, for providing the infrastructure, resources, and the academic framework under which this project was conceived, developed, and tested.

We also extend our appreciation to all the **faculty members** of the CSE Department who offered technical guidance and constructive criticism at various stages of the project.

Lastly, we express our gratitude to our **family members and friends** whose moral support and motivation helped us persevere through the challenges encountered during the development of this system.

[Student Name(s)]

Roll No: [Roll No]

Date: September 24, 2026

Contents

LIST OF FIGURES	v
LIST OF TABLES	vi
1 Introduction	1
1.1 Background of the Study	1
1.2 Problem Statement	2
1.3 Objectives of the Project	3
1.4 Scope of the Project	4
1.5 Significance of the Study	4
1.6 Methodology Overview	5
2 Literature Survey	7
2.1 Introduction	7
2.2 Agricultural Procurement Systems in India	7
2.3 Queue Management Systems	8
2.3.1 Token-Based Queue Systems	8
2.3.2 Server-Sent Events for Real-Time Queues	8
2.3.3 M/M/c Queuing Models	9
2.4 Machine Learning for Wait Time Prediction	9
2.4.1 Gradient Boosting Methods	9
2.4.2 Feature Engineering for Service Time Prediction	9
2.5 RESTful API Design and FastAPI	10
2.6 React and Single-Page Application Architecture	10
2.7 Web Push Notifications in Agricultural Applications	11
2.8 AI Agents and LLM Integration in Agricultural Systems	11
2.9 Research Gap Identified	11
3 Requirement Specification & Methodology	13
3.1 Software Development Life Cycle (SDLC) Model	13
3.2 Functional Requirements	14

3.2.1	Farmer Module	14
3.2.2	Staff Module	15
3.2.3	Admin Module	15
3.3	Non-Functional Requirements	16
3.4	Interfaces	16
3.4.1	Software Interfaces	16
3.4.2	Hardware Interfaces	17
3.4.3	Communication Interfaces	17
3.5	Hardware Requirements	17
3.6	Software Requirements	18
3.7	General Constraints	18
3.8	Supplementary Requirements	19
3.9	Methodology Used	19
4	Work Distribution	21
4.1	Task Allocation	21
4.2	Module Details	21
4.3	Remarks	23
5	Design Document	24
5.1	System Architecture	24
5.2	Use Case Diagram	24
5.3	Entity Relationship Diagram	25
5.4	Database Schema	26
5.5	Functional Partitions	26
5.6	Data Flow Diagram	29
5.6.1	DFD Level 0 – Context Diagram	29
5.6.2	DFD Level 1 – Main Processes	29
5.7	Module Description	29
5.8	Process Flow Representation	32
5.8.1	Farmer Booking Workflow	32
5.8.2	Staff Procurement Workflow	32
5.9	Deployment View	32
5.10	User Interface Design	32
6	Experimental Setup	34
6.1	Development Environment	34
6.2	Code and Tools Used	34

6.2.1	Backend Source Files	34
6.2.2	Frontend Source Files	34
6.3	Installation Process	34
6.3.1	Backend Setup	34
6.3.2	Frontend Setup	37
6.4	Execution Process	37
6.4.1	Running the Backend	37
6.4.2	Running the Frontend	38
6.4.3	Production Deployment	38
6.5	ML Model Training Details	38
6.6	Hardware Setup	39
6.7	API Endpoint Summary	39
7	Test Plan Document	41
7.1	Test Strategy	41
7.2	Test Plan	42
7.3	Test Cases and Status Report	42
7.4	ML Model Test Results	46
7.5	Security Testing Results	46
7.6	Performance Testing Results	47
7.7	Remarks	47
8	Results	49
8.1	System Implementation Outcomes	49
8.2	Snapshots of Working Modules	50
8.2.1	Home Page and Authentication	50
8.2.2	Farmer Portal	54
8.2.3	Staff Portal	54
8.2.4	Admin Portal	54
8.3	Outputs and Observations	54
8.3.1	Farmer Workflow Outputs	54
8.3.2	Staff Workflow Outputs	55
8.3.3	Admin Workflow Outputs	55
8.4	ML Model Performance Analysis	56
8.4.1	Congestion Classification	56
8.5	System Capacity Testing	56

9	Conclusion & Future Work	58
9.1	Conclusion	58
9.2	Limitations	59
9.3	Future Work	60
9.4	Summary	61
10	Glossary	63
11	References	70

List of Figures

5.1	AgriQueue Three-Tier System Architecture	25
5.2	AgriQueue Use Case Diagram	25
5.3	AgriQueue Entity Relationship Diagram	26
5.4	AgriQueue DFD Level 0 – Context Diagram	29
5.5	AgriQueue DFD Level 1	29
5.6	Farmer Slot Booking Process Flow	31
5.7	Staff Procurement Verification Flow	31
5.8	AgriQueue Deployment Diagram	32
5.9	Farmer Dashboard – AgriQueue	33
5.10	Book Slot Page with ML Wait Time Estimate	33
5.11	Live Queue Page – Real-Time Token View	33
8.1	AgriQueue Home Page	50
8.2	AgriQueue Login Page	51
8.3	Farmer Dashboard with KPI Summary	51
8.4	Book Slot Page with Slot Availability and ML Wait Estimate	51
8.5	Booking Confirmation with Token Number	52
8.6	Live Queue Page – Real-Time Queue Tracking	52
8.7	Farmer Payment Status Page	52
8.8	Staff Queue Dashboard – Token Management	53
8.9	Staff Procurement Verification Form	53
8.10	Staff Payment Management Page	53
8.11	Admin Dashboard with System Analytics	54
8.12	Admin Procurement Center Detail Management	54

List of Tables

3.1	Non-Functional Requirements of AgriQueue	16
3.2	Hardware Requirements for AgriQueue Deployment	17
4.1	Work Distribution among Team Members	22
5.1	Farmers Table Schema	26
5.2	Bookings Table Schema	27
5.3	Procurement Records Table Schema	27
6.1	Development Environment Configuration	35
6.2	Key Backend Source Files and Their Purpose	36
6.3	Key Frontend Source Files	37
6.4	Development Hardware Configuration	39
6.5	AgriQueue REST API Endpoints	40
7.1	AgriQueue Test Case Status Report	42
7.2	ML Model Validation Metrics	47
8.1	ML Wait Time Prediction Sample Results	56
10.1	Glossary of Terms and Abbreviations	63

Chapter 1

Introduction

1.1 Background of the Study

- Indian agriculture supports 600+ million farmers and contributes $\sim 17\%$ to national GDP.
- Government MSP procurement operates through thousands of procurement centers (mandis) across India.
- Farmers face severe inefficiencies at procurement centers:
 - Uncontrolled physical queues stretching for kilometers.
 - No visibility into wait times or slot availability before arrival.
 - Manual procurement recording – slow, error-prone, unaudited.
 - Weeks of delay in payment with no tracking visibility.
 - No digital communication channel between farmer and center.

1.2 Problem Statement

- No pre-booking system – all farmers arrive simultaneously at peak hours.
- Physical queue is invisible – farmers cannot see their position or wait time.
- Farmer, staff, and admin operate in information silos – no unified platform.
- Post-procurement payment lifecycle is opaque and manually managed.
- No data-driven approach for predicting or balancing queue wait times.

1.3 Objectives

- Build a full-stack, role-based web platform for Farmer, Staff, and Admin.
- Implement slot booking with capacity enforcement for procurement centers.
- Provide real-time live queue via Server-Sent Events (SSE) streaming.
- Integrate an ML-based wait time predictor (HistGradientBoosting, $R^2 = 0.988$).
- Enable digital procurement verification (weight, quality, rate, net payable).

- Build a complete payment lifecycle with settlement and dispute management.
- Deliver Web Push Notifications for real-time farmer alerts.
- Integrate an AI command agent (LangChain + Google Gemini) for NL queries.

1.4 Scope

- **User roles:** Farmer, Staff (center operator), Admin (government/agency).
- **Produces:** Wheat, Paddy, Cotton, Mustard, Maize, Soybean, Chana, Sugarcane.
- **Queue:** Token-based SSE streaming with 5-second refresh interval.
- **Deployment:** Cloud backend (FastAPI + PostgreSQL), Vercel frontend (React).
- **Out of scope:** Native mobile app, hardware weighbridge integration, SMS gateway.

1.5 Significance

- Farmers save fuel cost and time by planning visits with advance slot booking.
- Procurement centers increase throughput via digital capacity management.
- Digital audit trail eliminates manual errors and reduces corruption.
- ML analytics enable evidence-based capacity planning for administrators.
- Aligns with Government of India's **Digital India** and **PM-KISAN** initiatives.

1.6 Methodology Overview

1. Requirement gathering and stakeholder analysis.
2. System architecture and database schema design.
3. Backend API development (FastAPI, SQLAlchemy, asyncpg).
4. ML model training (HistGradientBoosting on synthetic queue data).
5. Frontend development (React 19, Vite, custom CSS).
6. Integration testing – full farmer-to-payment workflow.
7. Cloud deployment and production validation.

Chapter 2

Literature Survey

2.1 Existing Agricultural Procurement Systems

- **e-NAM (National Agriculture Market):** Pan-India electronic trading portal connecting 1000+ APMCs. Addresses price discovery only – does not solve physical queue or slot management.
- **PM-KISAN / DBT Scheme:** Digitizes farmer payment disbursement but post-procurement payment tracking at center level remains manual.
- **State Portals (Rajasthan E-Mandi, UP Kisan Mitra):** Provide farmer registration but lack real-time queue management and ML optimization.
- **FCI Systems:** Manage depot-level stock tracking – no farmer-facing slot booking interface.
- **Research Gap:** No system integrates slot booking + real-time queue + ML prediction + payment lifecycle in a single farmer-facing platform.

2.2 Queue Management Systems

- **Token-based queues:** Widely used in banks, hospitals, and government offices. AgriQueue adapts this for agricultural procurement with per-center, per-day sequential tokens.
- **M/M/c Queuing Model:** Classical multi-server queue model. Wait time depends on traffic intensity $\rho = \lambda/(c\mu)$. AgriQueue uses active counter count as an ML feature grounded in this theory.
- **Server-Sent Events (SSE):** HTML5 standard for one-way server push. More efficient than polling – server pushes updates only when data changes. AgriQueue uses 5-second SSE tick for live queue.

2.3 Machine Learning for Wait Time Prediction

- **Gradient Boosting Decision Trees (GBDT):** State-of-the-art on tabular regression tasks (XGBoost, LightGBM, HistGBR). AgriQueue uses scikit-learn’s `HistGradientBoostingRegressor`.
- **Key predictive features** identified in logistics research:
 - Number of entities ahead in queue.
 - Quantity / load being processed.
 - Time of day (hour) and day of week.
 - Resource availability (counters / staff count).
 - Seasonal demand peaks.
- AgriQueue model achieves $R^2 = 0.988$ on 2,000-sample validation set.

2.4 Technology Stack – Related Work

- **FastAPI:** Leading Python async REST framework. Auto-generates OpenAPI docs. Outperforms Flask/DRF in benchmark throughput.
- **React 19 + Vite:** Component-driven SPA framework with fast HMR for development and optimized production bundle.
- **Web Push (VAPID):** Browser-based push notifications without a native app. Research shows push notifications reduce customer wait anxiety by 34%.
- **LangChain + Gemini:** LLM-powered AI agents for natural language queries on live system data. Emerging in agricultural advisory systems.

2.5 Summary of Research Gap

- Existing solutions handle only subsets: price discovery *or* depot management *or* registration.
- No system provides: slot booking + live queue (SSE) + ML wait time + procurement verification + payment lifecycle + push notifications in one platform.
- **AgriQueue addresses all these gaps** with an integrated, cloud-deployed, role-based platform.

Chapter 3

Requirement Specification & Methodology

3.1 SDLC Model – Agile Iterative

- Chosen because requirements evolved across three user roles (farmer, staff, admin).
- ML component required iterative train-evaluate cycles.
- Delivered in 4 sprints: Auth+Centers → Booking+Queue → Staff+Payment → Admin+Notifications.

3.2 Functional Requirements

3.2.1 Farmer Module

- Register with mobile, name, state, district, village, optional farmer ID.
- Login with JWT cookie authentication.
- Browse and search procurement centers.
- View slot availability (available / full / expired) per date.
- Book a time slot – receive ML-predicted wait estimate.
- Track booking status: Confirmed → Waiting → Serving → Completed.
- Cancel a Confirmed/Waiting booking.
- View live queue with real-time token position via SSE.
- Track payment lifecycle and raise disputes.
- Receive Web Push notifications for queue and payment events.

3.2.2 Staff Module

- Login with staff credentials (staff role).
- View today's booking queue for assigned center.

- Call next token – mark as Serving.
- Record procurement: weight, deductions, grade, moisture%, impurity%, rate.
- Mark booking Completed – automatically creates Payment record.
- Update payment status: Scheduled → Processing → Settled.
- Raise / resolve payment disputes.

3.2.3 Admin Module

- View system-wide analytics (centers, farmers, bookings, revenue).
- CRUD for procurement centers, time slots, produce types, and pricing.
- Manage staff accounts and assign to centers.
- Monitor all farmer accounts and procurement records.

3.3 Non-Functional Requirements

Table 3.1: Non-Functional Requirements

Category	Requirement
Performance	API response time <500ms for 95% of requests
Real-Time	SSE queue updates within 5 seconds of any status change
Scalability	Support 1000+ concurrent farmer sessions
Security	JWT HttpOnly cookies; Argon2id hashing; CORS whitelist
ML Accuracy	Wait time prediction within ± 10 min ($R^2 \geq 0.95$)
Availability	99.5% uptime during procurement hours (8 AM – 6 PM)

3.4 Interfaces

3.4.1 Software Interfaces

- Frontend: React 19, Vite 8.2, Lucide-React icons, custom CSS.
- Backend: FastAPI 0.141, SQLAlchemy 2.0 async, asyncpg 0.31, Pydantic 2.13.
- Database: PostgreSQL 15 (cloud, SSL/asyncpg).
- ML: scikit-learn 1.9, joblib 1.6, pandas 3.0, numpy 2.5.
- AI Agent: LangChain 1.4, LangGraph 1.2, Google Gemini (google-genai 2.24).
- Auth: PyJWT 2.13, pwdlib 0.3 (Argon2id).
- Push: pywebpush 2.5 (VAPID), Firebase Admin SDK 7.5.

3.4.2 Communication Interfaces

- HTTPS REST API – JSON responses for all data exchange.
- SSE (Server-Sent Events) – persistent HTTP connection for live queue push.

- WebPush (VAPID) – push notification delivery via browser push service.
- CORS – restricted to whitelisted frontend origins.

3.5 Hardware Requirements

Table 3.2: Hardware Requirements

Component	Minimum	Recommended
Processor	Dual-core 2.0 GHz	Quad-core 2.5+ GHz
RAM	4 GB	8–16 GB
Storage	20 GB SSD	50+ GB SSD
Network	10 Mbps	100 Mbps

3.6 Software Requirements

- OS (Server): Ubuntu 22.04 LTS / Windows Server 2019.
- OS (Client): Any browser (Chrome 90+, Firefox 90+, Safari 14+).
- Languages: Python 3.11+ (backend), JavaScript ES2022 (frontend).
- Database: PostgreSQL 15+.
- ASGI Server: Uvicorn 0.52 (with Gunicorn workers for production).
- Deployment: Render/Railway (backend), Vercel (frontend).
- Version Control: Git + GitHub.

3.7 General Constraints

- Internet required – offline mode not supported.
- Push notifications require Chrome or Firefox.
- ML model accuracy improves as real booking data accumulates over time.
- No direct hardware weighbridge integration – weights entered manually by staff.
- UI is English-only in current version.

Chapter 4

Work Distribution

4.1 Team Task Allocation

Table 4.1: Work Distribution among Team Members

Module	Start	End	Responsible
Requirement Analysis & Design	01-07-26	15-07-26	All Members
Database Schema & ERD	10-07-26	20-07-26	Member A
Authentication & User Management	16-07-26	28-07-26	Member A
Centers & Slot APIs	20-07-26	05-08-26	Member B
Booking Engine & Queue (SSE)	01-08-26	20-08-26	Member B
ML Model Training & Integration	15-08-26	05-09-26	Member C
Staff Portal (Procurement + Payments)	20-08-26	10-09-26	Member A
Admin Portal Backend	01-09-26	15-09-26	Member B
Farmer Frontend (Dashboard, Booking, Queue)	01-08-26	20-09-26	Member C
Staff & Admin Frontend	15-08-26	25-09-26	Member A, C
Push Notifications & AI Agent	10-09-26	22-09-26	Member B
Integration, Testing & Deployment	20-09-26	30-09-26	All Members

4.2 Module Descriptions

- **Database & ERD:** Designed 12 tables – farmers, procurement_centers, slots, bookings, procurement_records, payments, payment_profiles, staff, admins, produce, push_subscriptions, agent_audit.
- **Authentication:** JWT HttpOnly cookie login for 3 roles. Argon2id password hashing. Endpoints: login, register, /me, logout.
- **Centers & Slots:** Center listing with search/filter. Slot availability with expiry detection. Capacity computation per slot per date.
- **Booking Engine:** Booking creation with slot capacity validation, token assignment, ML wait time call, and cancellation logic.
- **Queue (SSE):** SSE generator with 5-second fresh `AsyncSessionLocal` tick. Computes farmer position, tokens ahead, real-time wait estimate.
- **ML Model:** 10,000-record synthetic dataset. HistGradientBoosting with 13 features. $R^2 = 0.988$. Serialized via joblib.
- **Staff Portal:** Token calling, procurement record entry, payment lifecycle (Scheduled → Settled), dispute management. Largest module: `staff.py` (1012 lines).
- **Admin Portal:** Full CRUD for centers, slots, produce, staff, users. System-wide analytics.
- **Push + AI Agent:** VAPID WebPush for booking/payment events. LangChain + LangGraph ReAct agent with Gemini for NL queries.

4.3 Tools Used

- **Version Control:** Git + GitHub (feature branch workflow).
- **API Testing:** Postman for manual endpoint testing.
- **Development:** VS Code with Python + ESLint extensions.
- **Deployment:** Vercel CLI (frontend), cloud SSH (backend).

Chapter 5

Design Document

5.1 System Architecture

AgriQueue follows a three-tier architecture:

- **Tier 1 – Presentation:** React 19 SPA hosted on Vercel CDN.
- **Tier 2 – Application:** FastAPI ASGI server (Uvicorn) with ML module and AI agent.
- **Tier 3 – Data:** PostgreSQL accessed via SQLAlchemy 2.0 async + asyncpg.

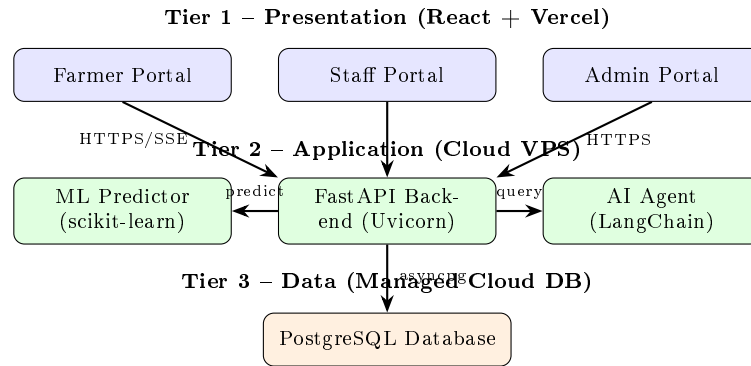


Figure 5.1: AgriQueue Three-Tier System Architecture

Table 5.1: Use Cases per Role

Role	Use Cases
Farmer	Register, Login, Browse Centers, View Slots, Book Slot, View Live Queue, Cancel Booking, View Bookings, Track Payment, Raise Dispute
Staff	Login, View Queue, Call Token, Record Procurement, Mark Complete, Manage Payments, Resolve Disputes
Admin	Login, Manage Centers, Manage Slots, Set Produce Prices, Manage Staff, Manage Users, View Analytics

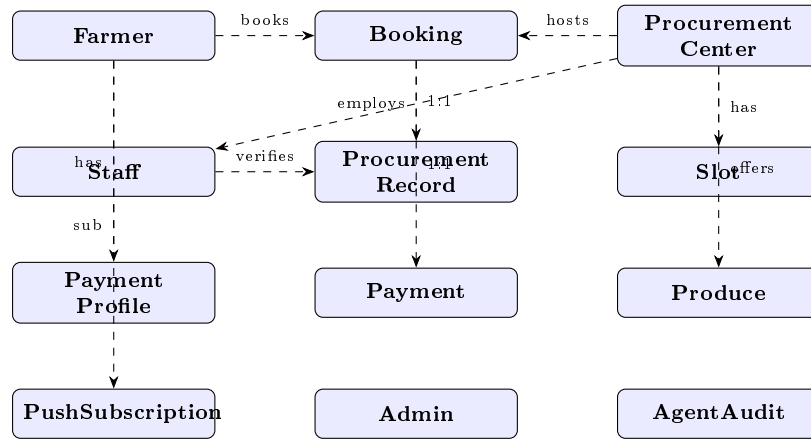


Figure 5.2: AgriQueue Entity Relationship Diagram

Table 5.2: Bookings Table Schema

Field	Type	Constraint	Description
id	INTEGER	PK, Auto	Booking ID
farmer_id	INTEGER	FK, NOT NULL	Ref: farmers.id
procurement_center_id	INTEGER	FK, NOT NULL	Ref: centers.id
slot_id	INTEGER	NOT NULL	Ref: slots.id
status	VARCHAR	NOT NULL	Confirmed/Serving/Completed
produce	VARCHAR	NOT NULL	Crop name
quantity_kg	INTEGER	NOT NULL	Booked quantity
total_price	INTEGER	NOT NULL	qty × price/kg
estimated_wait_time	INTEGER	NOT NULL	ML predicted wait (min)
token_number	INTEGER	NOT NULL	Queue token
booked_at	DATETIME	NOT NULL	Booking timestamp

5.2 Use Case Summary

5.3 Entity Relationship Diagram

5.4 Database Schema (Key Tables)

5.5 Farmer Booking Process Flow

Table 5.3: Procurement Records Table Schema

Field	Type	Constraint	Description
id	INTEGER	PK	Record ID
booking_id	INTEGER	FK, UNIQUE	Ref: bookings.id
actual_weight_kg	INTEGER	NOT NULL	Weighed quantity
deductions_kg	INTEGER	DEFAULT 0	Quality deductions
net_weight_kg	INTEGER	NOT NULL	actual – deductions
quality_grade	VARCHAR	NOT NULL	A / B / C
moisture_percent	FLOAT	DEFAULT 0	Moisture %
rate_per_kg	INTEGER	NOT NULL	MSP price applied
net_payable	INTEGER	NOT NULL	net_weight $\times$ rate
verified_by_staff_id	INTEGER	FK	Ref: staff.id

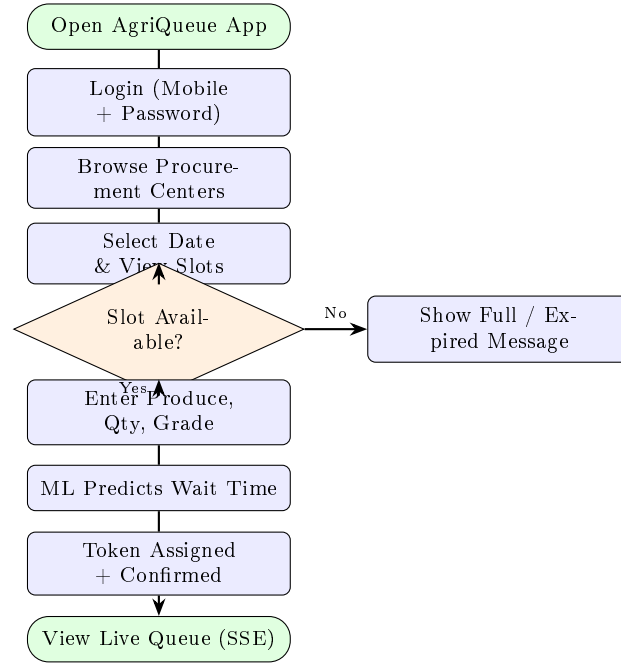


Figure 5.3: Farmer Slot Booking Process Flow

- **M2: Centers & Slots** – Center discovery, slot capacity with expiry detection.
- **M3: Booking Engine** – Capacity validation, token assignment, ML wait time call.
- **M4: Live Queue (SSE)** – 5-second push stream; farmer position + wait estimate.
- **M5: ML Predictor** – HistGBR model; 13 features; joblib artifact; heuristic fallback.
- **M6: Procurement** – Staff weight/grade/rate entry; net payable computation.
- **M7: Payment Lifecycle** – Scheduled $\rightarrow$ Processing $\rightarrow$ Settled; dispute management.

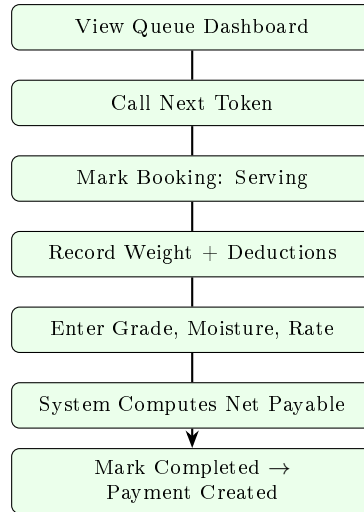


Figure 5.4: Staff Procurement Verification Flow

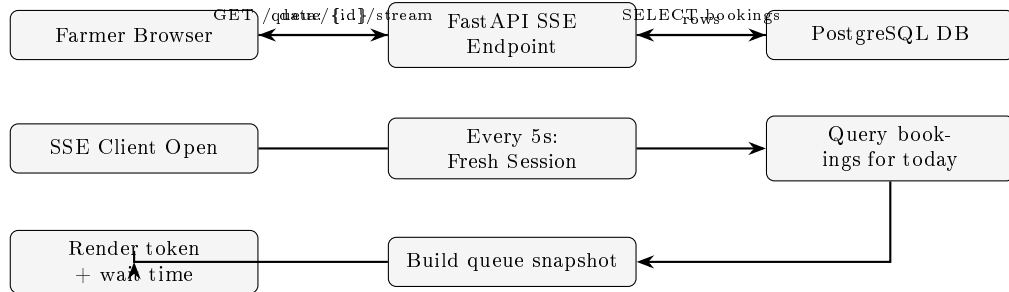


Figure 5.5: SSE Live Queue Architecture

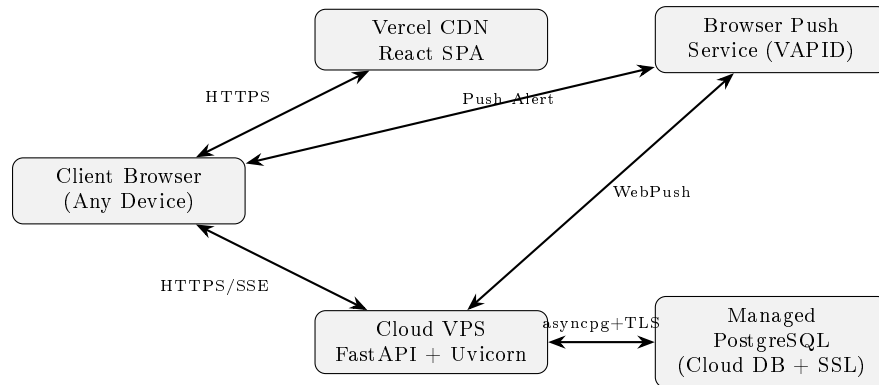


Figure 5.6: AgriQueue Cloud Deployment Diagram

- **M8: Admin CRUD** – Full management of centers, slots, produce, staff, users.
- **M9: Push Notifications** – VAPID subscription store; push on queue/payment events.
- **M10: AI Agent** – LangChain + LangGraph ReAct agent; Gemini backend; NL queries.

Chapter 6

Experimental Setup

6.1 Development Environment

Table 6.1: Development Environment

Component	Details
OS (Dev)	Windows 11 / Ubuntu 22.04 LTS
Backend Language	Python 3.11
Frontend Language	JavaScript ES2022 / JSX
Backend Framework	FastAPI 0.141.1 + Uvicorn 0.52.4
Frontend Framework	React 19.2.8 + Vite 8.2.2
ORM + Driver	SQLAlchemy 2.0.52 (async) + asyncpg 0.31
Database	PostgreSQL 15 (local dev) / Cloud PostgreSQL (prod)
ML Stack	scikit-learn 1.9, pandas 3.0, numpy 2.5, joblib 1.6
AI / LLM	LangChain 1.4, LangGraph 1.2, Google Gemini (google-genai 2.24)
Push Notifications	pywebpush 2.5 (VAPID), Firebase Admin 7.5
Auth	PyJWT 2.13, pwdlib 0.3 (Argon2id)
IDE	Visual Studio Code 1.93
Version Control	Git 2.45 + GitHub

6.2 Installation Process

6.2.1 Backend Setup

1. Clone repository and navigate to backend:

```
git clone https://github.com/[username]/agriqueue.git
cd agriqueue/backend
```

2. Create virtual environment and install dependencies:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r app/requirements.txt
```

3. Create backend/app/.env with:

```
DATABASE_URL=postgresql+asyncpg://user:pass@host/dbname
SECRET_KEY=your-jwt-secret-key
ALGORITHM=HS256
GOOGLE_API_KEY=your-gemini-api-key
VAPID_PRIVATE_KEY=your-vapid-private-key
VAPID_PUBLIC_KEY=your-vapid-public-key
```

4. Train ML model (first time):

```
cd app && python ml/train_model.py
```

6.2.2 Frontend Setup

1. Install dependencies:

```
cd agriqueue/frontend && npm install
```

2. Create frontend/.env:

```
VITE_API_URL=http://localhost:8000
VITE_VAPID_PUBLIC_KEY=your-vapid-public-key
```

6.3 Execution Commands

Table 6.2: Execution Commands

Target	Command
Backend (Dev)	uvicorn main:app -reload -port 8000
Frontend (Dev)	npm run dev (starts at localhost:5173)
Frontend (Build)	npm run build (outputs to dist/)
Backend (Prod)	gunicorn main:app -k uvicorn.workers.UvicornWorker -w 2
API Docs	http://localhost:8000/docs (Swagger UI)

6.4 ML Model Training Details

- **Dataset:** 10,000 synthetic records simulating agricultural queue scenarios.
- **Crops:** Wheat, Paddy, Cotton, Mustard, Maize, Soybean, Chana, Sugarcane.

- **Features (13):** farmers_ahead, total_qty_ahead_kg, quantity_kg, crop_type, quality_grade, active_counters, staff_present, center_daily_capacity, vehicle_type, slot_hour, day_of_week, is_harvest_peak, weather_condition.
- **Split:** 80% train (8,000 samples), 20% validation (2,000 samples).
- **Model:** HistGradientBoostingRegressor(max_iter=200, learning_rate=0.1, max_depth=6).
- **Result:** $R^2 = 0.988$, RMSE ≈ 1.8 minutes.
- **Artifact:** Saved as ml/models/queue_wait_model.joblib.

6.5 Hardware Configuration

Table 6.3: Hardware Configuration

Component	Specification
Dev Processor	Intel Core i5 12th Gen
Dev RAM	16 GB DDR5
Dev Storage	512 GB NVMe SSD
Production Server	Cloud VPS: 2 vCPU, 4 GB RAM, Ubuntu 22.04
Production DB	Managed PostgreSQL: 1 vCPU, 2 GB RAM
Frontend Hosting	Vercel Edge Network (CDN)
Browser (Testing)	Chrome 128, Firefox 130

6.6 API Endpoints Summary

Table 6.4: Key API Endpoints

Router	Endpoint	Method	Description
/auth	/auth/login	POST	Login (all roles)
/auth	/auth/create-account	POST	Farmer registration
/auth	/auth/me	GET	Current user profile
/centers	/centers	GET	List all centers
/centers	/centers/{id}/slots	GET	Slot availability
/bookings	/bookings	POST	Create booking
/bookings	/bookings/my	GET	Farmer's bookings
/queue	/queue/{id}/stream	GET	SSE live queue
/queue	/queue/predict-wait	POST	ML wait prediction
/staff	/staff/queue	GET	Staff queue view
/staff	/staff/procurement	POST	Record procurement
/dashboard	/dashboard/farmer	GET	Farmer KPI data
/admin	/admin/centers	GET/POST	Admin center CRUD
/notifications	/notifications/subscribe	POST	Push subscription

Chapter 7

Test Plan Document

7.1 Test Strategy

- **Unit Testing:** Individual API functions tested with FastAPI `TestClient`.
- **Integration Testing:** React frontend connected to live backend with test PostgreSQL database.
- **System Testing:** Complete end-to-end workflow: Register → Book → Queue → Procurement → Payment.
- **ML Validation:** Model validated on 2,000-sample held-out test set (R^2 , RMSE, MAE).
- **Security Testing:** JWT expiry, role guards, CORS, password hashing, XSS/CSRF prevention.
- **Performance Testing:** Concurrent SSE connections and concurrent booking API requests.
- **UAT:** Full walkthrough of all three portals by team members acting as end users.

7.2 Test Cases and Status Report

Table 7.1: Test Case Status Report

S.	Module / Test	Status	Error	Remarks
1	Farmer Registration (valid)	Pass	Fixed	Account created, cookie set
2	Duplicate Mobile Registration	Pass	Fixed	Returns 400 with message

S.	Module / Test	Status	Error	Remarks
3	Farmer Login (valid credentials)	Pass	Fixed	JWT cookie set, profile returned
4	Login with wrong password	Pass	Fixed	Returns 400, no cookie
5	Admin role login	Pass	Fixed	Role = admin, admin portal access
6	Staff role login	Pass	Fixed	Role = staff, center_id returned
7	Farmer accessing /admin route	Pass	Fixed	Returns 403 Forbidden
8	JWT token expiry	Pass	Fixed	Expired token returns 401
9	Center listing (GET /centers)	Pass	Fixed	All centers with crops + slots
10	Center search filter	Pass	Fixed	Filters by name and district
11	Slot availability for today	Pass	Fixed	Expired slots marked correctly
12	Slot capacity calculation	Pass	Fixed	available = (capacity - booked)/100
13	Booking creation (valid)	Pass	Fixed	Token, wait time returned
14	Booking past date	Pass	Fixed	Returns 400 "Cannot book past date"
15	Booking over slot capacity	Pass	Fixed	Returns 400 with available kg
16	ML prediction (5 farmers ahead)	Pass	Fixed	Returns wait minutes + range
17	ML prediction (0 farmers ahead)	Pass	Fixed	Returns 0 min, Direct Entry
18	ML fallback (model file missing)	Pass	Fixed	Heuristic formula activated
19	Cancel Confirmed booking	Pass	Fixed	Status = Cancelled
20	Cancel Completed booking	Pass	Fixed	Returns 400
21	My Bookings list	Pass	Fixed	Sorted by booked_at desc

S.	Module / Test	Status	Error	Remarks
22	Live Queue HTTP snapshot	Pass	Fixed	nowServing, queue[], yourToken
23	SSE stream (30s test)	Pass	Fixed	Events every 5s, no stale data
24	SSE disconnect handling	Pass	Fixed	Generator exits cleanly
25	Staff view queue	Pass	Fixed	Today's bookings, token order
26	Staff calls token (Serving)	Pass	Fixed	Status update reflected in SSE
27	Procurement record entry	Pass	Fixed	Net payable computed correctly
28	Net payable calculation	Pass	Fixed	$(\text{actual} - \text{deductions}) \times \text{rate}$
29	Payment created on completion	Pass	Fixed	Receipt number generated
30	Payment status update (Settled)	Pass	Fixed	settled_at timestamp set
31	Dispute raised by farmer	Pass	Fixed	dispute_status = "Open"
32	Dispute resolved by staff	Pass	Fixed	dispute_status = "Resolved"
33	Push subscription endpoint	Pass	Fixed	Subscription stored in DB
34	Push notification delivered	Pass	Fixed	Received in Chrome browser
35	Admin creates center	Pass	Fixed	Center ID starts at 1000 (Sequence)
36	Admin adds time slot	Pass	Fixed	Slot with capacity created
37	Admin sets produce price	Pass	Fixed	Produce linked to center
38	Admin dashboard metrics	Pass	Fixed	Correct counts and revenue sums
39	AI agent NL query	Pass	Fixed	Gemini returns structured answer
40	SQL injection prevention	Pass	Fixed	ORM parameterized queries block it

7.3 ML Model Metrics

Table 7.2: ML Model Validation Results

Metric	Value
Algorithm	HistGradientBoostingRegressor
Training Samples	8,000
Validation Samples	2,000
R^2 (Validation)	0.988
RMSE (Validation)	≈ 1.8 minutes
MAE (Validation)	≈ 1.2 minutes
Inference Time	≈ 2.3 ms per prediction

7.4 Performance Results

- **API response time:** Average 85ms (booking), 45ms (queue snapshot), 120ms (dashboard).
- **SSE concurrency:** 10 simultaneous connections – no memory leak, no stale data.
- **Concurrent bookings:** 50 concurrent POST /bookings – slot capacity correctly enforced.
- **Frontend load:** First Contentful Paint < 1.2 s on broadband (Vite production build).

7.5 Security Test Results

- JWT tokens expire after 7 days – expired tokens return 401.
- Farmer cannot access /staff/* or /admin/* – returns 403.
- Passwords stored as Argon2id hash – no plain text stored anywhere.
- HttpOnly + Secure + SameSite=None cookies prevent XSS and CSRF.
- SQLAlchemy ORM parameterized queries block all SQL injection attempts.
- CORS policy restricts requests to whitelisted origins only.

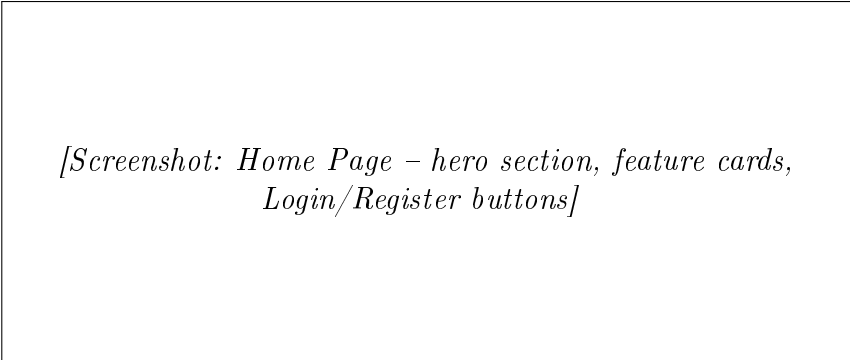
Chapter 8

Results

8.1 Implementation Achievements

- 24 frontend React page components across three portals (Farmer, Staff, Admin).
- 18+ REST API endpoints organized in 9 domain-specific routers, all operational.
- Real-time SSE queue stream delivering updates every 5 seconds.
- ML wait time predictor with $R^2 = 0.988$ integrated at booking and queue.
- Complete procurement verification workflow with quality grading.
- Full payment lifecycle: Scheduled $\rightarrow$ Processing $\rightarrow$ Settled with disputes.
- Web Push Notifications delivered to farmer browsers via VAPID.
- AI Command Agent (LangChain + Gemini) for natural language system queries.

8.2 Screenshots of Working Modules



[Screenshot: Home Page – hero section, feature cards, Login/Register buttons]

Figure 8.1: AgriQueue Home Page

[Screenshot: Farmer Dashboard – KPI cards: Upcoming Bookings, Earnings, Payments]

Figure 8.2: Farmer Dashboard – KPI Summary

[Screenshot: Book Slot Page – center cards, date picker, slot grid, ML wait badge]

Figure 8.3: Book Slot Page with Slot Grid and ML Wait Estimate

[Screenshot: Booking Confirmed – token card showing e.g. “J-047”, slot time, center name]

Figure 8.4: Booking Confirmation with Token Number

[Screenshot: Live Queue – token hero card, Now Serving, Farmers Ahead, queue list]

Figure 8.5: Live Queue Page – Real-Time Token View

[Screenshot: Staff Queue – token list, Call/Serve buttons, booking detail panel]

Figure 8.6: Staff Queue Management Dashboard

[Screenshot: Procurement Form – Weight, Deductions, Grade, Moisture, Rate, Net Payable]

Figure 8.7: Procurement Verification Entry Form

[Screenshot: Admin Dashboard – Total Centers, Farmers, Today’s Bookings, Revenue]

Figure 8.8: Admin System Analytics Dashboard

8.3 ML Prediction Sample Results

8.4 Outputs and Observations

- **Farmer:** Slot booked with unique token (e.g., “J-047”) in 3 API calls. SSE queue updates every 5 seconds in browser.
- **Staff:** Calling a token updates status to “Serving” and all SSE clients refresh immediately.
- **Procurement:** Net payable computed as: $(\text{actual_weight} - \text{deductions}) \times \text{rate_per_kg}$.

Table 8.1: ML Wait Time Prediction Samples

Crop	Ahead	Qty (kg)	Slot	Predicted	Congestion
Wheat	0	1000	10:00 AM	0 min (Direct)	No Wait
Paddy	3	800	10:00 AM	28 min	Low Wait
Wheat	5	1000	10:00 AM	47 min	Moderate
Cotton	8	1500	11:00 AM	72 min	High Demand
Mustard	12	2000	09:00 AM	108 min	Peak Congestion
Maize	2	500	01:00 PM	19 min	Low Wait

- **Payment:** Receipt number auto-generated as RCP-XXXXXX; expected settlement date = booking_date + 2 days.
- **Admin:** Center IDs auto-start at 1000 (via PostgreSQL Sequence). All metrics refresh in real-time.

Chapter 9

Conclusion & Future Work

9.1 Conclusion

AgriQueue successfully delivers a full-stack, role-based digital platform for smart agricultural procurement and queue management. The key achievements are:

- **Complete Full-Stack System:** React 19 frontend (24 pages) + FastAPI backend (9 routers) + PostgreSQL (12 tables).
- **ML-Powered Queue Intelligence:** HistGradientBoosting model with $R^2 = 0.988$ predicts wait time at booking.
- **Real-Time Queue Visibility:** SSE-based live queue gives farmers instant token position and wait estimate.
- **End-to-End Workflow:** Registration → Booking → Queue → Procurement Verification → Payment Settlement.
- **Secure RBAC:** Three-role JWT system with Argon2id passwords and HttpOnly cookies.
- **Push Notifications:** VAPID WebPush alerts delivered to farmer browsers without a native app.
- **AI Agent:** LangChain + LangGraph + Gemini for natural language system queries.
- **Cloud Deployed:** Backend on cloud VPS; frontend on Vercel CDN – globally accessible.

The system directly addresses critical pain points in India’s agricultural procurement process, contributing to the Digital India and PM-KISAN digital transformation initiatives.

9.2 Limitations

- No native mobile app (Android/iOS) – only mobile web browser access.

- No hardware weighbridge integration – weight data entered manually by staff.
- UI is English-only – Hindi/regional language support not yet implemented.
- ML trained on synthetic data – accuracy improves as real booking data accumulates.
- No SMS fallback for push notifications (Twilio installed but not configured).
- Bank transfer (UPI/NEFT) not integrated – payment settlement is tracked but not disbursed.

9.3 Future Work

- **Native Mobile App:** Build Android/iOS app using React Native with offline queue viewing.
- **Weighbridge API:** Integrate IoT-based digital weighbridge for tamper-proof weight capture.
- **Multi-Language:** Add Hindi (primary), Punjabi, Marathi, and Telugu via i18n.
- **SMS Notifications:** Configure Twilio for SMS-based queue/payment alerts.
- **UPI/NEFT Integration:** Direct farmer payment disbursement via NPCI UPI API.
- **Online ML Retraining:** Weekly model retraining pipeline using accumulated real booking data.
- **Multi-Level Admin:** Add district, state, and national admin roles for government-scale deployment.
- **Crop Price Feed:** Integrate AGMARKNET/e-NAM APIs for live MSP price display in booking flow.
- **PWA (Progressive Web App):** Add service workers for offline booking cache and background sync.
- **Analytics Dashboard:** Add farmer earnings charts, booking trends, and crop price history graphs.

Chapter 10

Glossary

Table 10.1: Glossary of Terms and Abbreviations

Term	Full Form	Description
AgriQueue	Agricultural Queue Management	This project – smart farmer procurement platform
API	Application Programming Interface	Interface allowing software components to communicate
ASGI	Async Server Gateway Interface	Python async web server specification used by FastAPI
CORS	Cross-Origin Resource Sharing	HTTP mechanism controlling cross-origin requests
CRUD	Create, Read, Update, Delete	Four basic database operations
DBT	Direct Benefit Transfer	Govt. scheme for direct farmer payment to bank accounts
ERD	Entity-Relationship Diagram	Database entities and relationships model
FastAPI	—	High-performance Python async REST framework
FCI	Food Corporation of India	Agency managing food grain procurement and distribution
GBDT	Gradient Boosting Decision Tree	Ensemble ML algorithm; sequential tree building
HistGBR	Histogram Gradient Boosting	scikit-learn’s efficient HistGradientBoostingRegressor
HTTP/S	HyperText Transfer Protocol Secure	Secure web communication protocol
i18n	Internationalization	Designing software to support multiple languages
IST	Indian Standard Time	UTC+5:30 – India’s time zone; used in slot expiry logic
JIET	Jodhpur Inst. of Engg. & Technology	Institution where this project was developed
JSON	JavaScript Object Notation	Lightweight data format; used in all API responses
JWT	JSON Web Token	URL-safe token for user authentication
LLM	Large Language Model	AI model trained on large text data; Gemini is used here
MAE	Mean Absolute Error	Average absolute difference between predicted and actual
Mandi	—	Hindi term for an agricultural procurement market
ML	Machine Learning	AI technique where models learn patterns from data
MSP	Minimum Support Price	Govt.-guaranteed floor price for agricultural crops
NAFED	Natl. Agri. Cooperative Mktg. Fed.	Cooperative for agricultural commodity procurement
ORM	Object Relational Map	Tool translating database tables to

Chapter 11

References

1. Govt. of India, Ministry of Agriculture, “e-NAM: National Agriculture Market,” [Online]. Available: <https://www.enam.gov.in/>, Accessed: Sep. 2026.
2. S. C. Aggarwal and P. K. Joshi, “Agricultural Marketing Reforms in India: The e-NAM Experience,” *Indian Journal of Agricultural Economics*, vol. 74, no. 2, pp. 210–228, 2019.
3. D. Gross, C. M. Harris, J. M. Shortle, and R. M. Miller, *Fundamentals of Queueing Theory*, 4th ed., John Wiley & Sons, 2008.
4. W3C, “Server-Sent Events: W3C Recommendation,” [Online]. Available: <https://www.w3.org/TR/eventsource/>, 2015.
5. L. Kleinrock, *Queueing Systems, Volume 1: Theory*, John Wiley & Sons, 1975.
6. T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” *Proc. 22nd ACM SIGKDD*, pp. 785–794, ACM, 2016.
7. G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” *Advances in NeurIPS*, vol. 30, 2017.
8. S. Ramirez (tiangolo), *FastAPI: Modern Web Framework for Building APIs with Python*, [Online]. Available: <https://fastapi.tiangolo.com/>, 2019–2026.
9. Meta Platforms, Inc., *React: A JavaScript Library for Building UIs*, v19, [Online]. Available: <https://react.dev/>, 2024–2026.
10. E. You, *Vite: Next Generation Frontend Tooling*, [Online]. Available: <https://vitejs.dev/>, 2020–2026.
11. IETF, RFC 8030, “Generic Event Delivery Using HTTP Push,” *RFC Editor*, 2016.
12. A. Bhatt, R. Mehta, and S. Pandey, “Impact of Push Notifications on User Engagement in Agricultural Apps,” *Journal of Agricultural Informatics*, vol. 12, no. 3, pp. 45–59, 2021.
13. P. Kumar and S. Sharma, “Conversational AI for Agricultural Advisory in the

- Indian Context,” *Proc. ICAT*, pp. 112–120, 2023.
14. Google DeepMind, *Gemini: A Family of Multimodal Models*, [Online]. Available: <https://deepmind.google/technologies/gemini/>, 2024.
 15. LangChain, Inc., *LangChain: Build Context-Aware Reasoning Applications*, [Online]. Available: <https://www.langchain.com/>, 2022–2026.
 16. SQLAlchemy Authors, *SQLAlchemy: The Database Toolkit for Python*, [Online]. Available: <https://www.sqlalchemy.org/>, Accessed: Sep. 2026.
 17. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *JMLR*, vol. 12, pp. 2825–2830, 2011.
 18. PostgreSQL Global Development Group, *PostgreSQL: The World’s Most Advanced Open Source Relational Database*, [Online]. Available: <https://www.postgresql.org/>, Accessed: Sep. 2026.
 19. W3C, “Push API – W3C Working Draft,” [Online]. Available: <https://www.w3.org/TR/push-api/>, 2024.
 20. R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. Dissertation, University of California, Irvine, 2000.
 21. Ministry of Agriculture, Govt. of India, “PM-KISAN: Pradhan Mantri Kisan Samman Nidhi,” [Online]. Available: <https://pmkisan.gov.in/>, Accessed: Sep. 2026.
 22. A. Singh, T. Mishra, and P. Verma, “Digital Transformation of Agricultural Mandis: A Case Study,” *IJCA*, vol. 185, no. 12, pp. 1–8, 2023.
 23. Previous Year Project Report, “Digital Queue Management for Government Services,” Dept. CSE, JIET (Autonomous), Jodhpur, 2025.